German Accident Open Data Explorer

10-Minute Presentation Spoken Script & Oral Defense Guide

Student Presenter: Samay Patel (Matriculation No. 916936)

Degree Program: M.Sc. Web Engineering / Automotive Software

Academic Term: DBW Project · TU Chemnitz 2026

Examiners: Prof. Dr. Michael Martin & Florian Hahn

⏱️ **MINUTE-BY-MINUTE TALK TIMELINE (10 MINUTES TOTAL):**

```
[0:00 - 0:40] Slide 1: Introduction & Welcome
[0:40 - 1:10] Slide 2: Agenda Outline
[1:10 - 1:55] Slide 3: Problem vs. Solution & Core Objectives
[1:55 - 2:40] Slide 4: 3-Tier System Architecture & Tech Stack
[2:40 - 3:25] Slide 5: Data Pipeline & Ingestion Execution (import_data.py)
[3:25 - 4:10] Slide 6: PostgreSQL Schema & 8-Digit AGS Key Harmonization
[4:10 - 4:55] Slide 7: FastAPI REST API Architecture & 7 Endpoints
[4:55 - 5:35] Slide 8: Glassmorphism Web SPA & Modal Response Viewer
[5:35 - 6:15] Slide 9: Interactive OpenAPI / Swagger Documentation (/docs)
[6:15 - 8:00] Slide 10: Live Application & API Walkthrough [DEMO]
[8:00 - 8:45] Slide 11: Technical Challenges & Key Design Decisions
[8:45 - 9:30] Slide 12: Summary & Future Roadmap
[9:30 - 10:00] Slide 13: Conclusion & Opening Oral Defense Q&A
```

Slide 1: Title Slide & Introduction

`0:00 - 0:40`

Goal: Professional greeting, introducing yourself, course name, and thesis core statement.

> "Good morning, Prof. Dr. Martin and Mr. Hahn.
>
> My name is **Samay Patel**, matriculation number **916936**, studying Master's in Web Engineering at TU Chemnitz.
> Today, I am excited to present my term project for Datenbanken und Web-Techniken titled **'German Accident Open Data Explorer'**.
>
> In this project, I built an automated ETL data pipeline and a full 3-tier web service that harmonizes heterogeneous official German open data from the Unfallatlas accident register and Destatis regional statistics."

Slide 2: Presentation Agenda

`0:40 - 1:10`

Goal: Briefly outline the structure of the talk.

> "Here is a quick overview of what we will cover today.
>
> We will look at the core problem with raw open data, our 3-tier system architecture, the automated ingestion script, PostgreSQL database schema, FastAPI REST endpoints, the dark glassmorphic frontend, and finally, a live walkthrough of all examiner verification queries before wrapping up with key technical takeaways."

Slide 3: Project Overview & Objectives

`1:10 - 1:55`

Goal: Explain why raw German open data is fragmented and how your system solves it.

> "When working with official German open data, we face three main challenges:
> First, **structural heterogeneity**—Unfallatlas provides individual point collision logs, while Destatis provides demographic census sheets.
> Second, **key inconsistencies**—regional keys vary in length from 2-digit state codes to 8-digit municipal codes.
> Third, **locale issues**—coordinates are often stored using German decimal commas like '13,73726', which crashes standard float parsers.
>
> My project solves this by standardizing all records into uniform 8-digit AGS foreign keys, cleaning coordinates, and serving dynamic metrics through 7 REST routes over PostgreSQL with zero hardcoded values."

Slide 4: Tech Stack & 3-Tier Architecture

`1:55 - 2:40`

Goal: Demonstrate clean multi-tier web application architecture.

> "Our system follows a strict 3-tier web application architecture:
> 1. Raw ZIP archives are processed by our automated Python script `import_data.py`.
> 2. Structured metrics are stored in a local PostgreSQL relational database.
> 3. FastAPI exposes async REST endpoints using `psycopg2` and `RealDictCursor`.
> 4. Our Single Page Application queries FastAPI using asynchronous JavaScript `fetch()` requests without refreshing the page.
>
> The frontend never connects directly to PostgreSQL, ensuring clean tier separation."

Slide 5: Automated Data Pipeline

`2:40 - 3:25`

Goal: Explain step-by-step ETL ingestion and foreign key safety logic.

> "The ingestion pipeline inside `import_data.py` executes in 7 automated steps:
> It unzips raw archives, initializes DDL tables, inserts legal provenance metadata for `dl-de/by-2-0` licensing, parses Destatis population baselines, and harmonizes accident records.
>
> Crucially, columns `ULAND`, `UREGBEZ`, `UKREIS`, and `UGEMEINDE` are concatenated using `.zfill()` into 8-digit strings.
> To guarantee foreign key safety, if an accident references a municipal code missing from Destatis, the script auto-creates a fallback regional entry before inserting."

Slide 6: PostgreSQL Database Schema

`3:25 - 4:10`

Goal: Explain relational star schema and 8-digit AGS matching strategy.

> "Our database uses a normalized relational star schema:
> - The **`accidents` fact table** stores event metrics like year, month, hour, severity category, lighting, participant flags for cars, bikes, and pedestrians, along with numeric WGS84 floats.
> - The **`regions` dimension table** stores the 8-digit `ags` primary key, region name, level, and population.
>
> For example, Dresden Stadt is stored as `'14612000'`. Using this padded format allows us to execute instant state-wide SQL wildcard queries using `ags LIKE '14%'` for Saxony."

Slide 7: REST API Architecture & Endpoints

4:10 - 4:55

Goal: Highlight dynamic query execution and endpoint mapping.

> "Our backend engine built with FastAPI exposes 7 REST routes.
> Routes include `/metadata/earliest-year` for global timeline boundaries, `/aggregates/accidents` for dynamic filtering by AGS, year, and pedestrian flags, `/regions` for regional metadata, and custom routes for grader bonus questions.
>
> Every route runs dynamic SQL queries against PostgreSQL at request time, ensuring 100% dynamic calculation without static caching."

Slide 8: Glassmorphism Web SPA & Modal Viewer

4:55 - 5:35

Goal: Explain frontend aesthetics and raw JSON overlay modal.

> "The frontend is designed with dark mode glassmorphism using Tailwind CSS.
> Clicking any scenario card triggers an asynchronous `fetch()` call using Vanilla JavaScript.
>
> The response is rendered inside a slide-over JSON overlay modal, showing the exact calculated numbers along with official `dl-de/by-2-0` legal provenance tags for complete transparency."

Slide 9: Interactive API Documentation (/docs)

5:35 - 6:15

Goal: Showcase OpenAPI auto-documentation and Pydantic validation.

> "FastAPI automatically generates interactive OpenAPI 3.0 documentation at `/docs`.
> We use Pydantic response models like `AccidentMetricsResponse` to enforce strict schema types.
>
> Furthermore, we implemented parameter guards—if an invalid AGS length is provided, the API returns a structured HTTP 400 Bad Request error."

Slide 10: Live Demonstration Roadmap [LIVE DEMO]

`6:15 - 8:00`

Goal: Switch to browser tabs (http://127.0.0.1:8000/) and test all scenarios live.

> "Now, let's look at the live running application at `http://127.0.0.1:8000/`.
>
> 1. *(Click Scenario 1 Card)* → Clicking Scenario 1 queries `/metadata/earliest-year`, returning **2023** as our earliest year.
> 2. *(Click Scenario 2 Card)* → Scenario 2 calculates personal injury accidents in Saxony for 2023 (`ags=14&year=2023`), returning **11,842** incidents.
> 3. *(Click Scenario 3 Card)* → Scenario 3 filters pedestrian collisions in Berlin for 2023 (`ags=11`), returning **1,656** records.
> 4. *(Click Bonus A Card)* → Bonus Scenario A queries bicycle collisions in Dresden for 2024 (`ags LIKE '14612%' AND ist_rad = 1`), returning **1,009** cases.
> 5. *(Click Bonus B Card)* → Bonus Scenario B executes dynamic string slicing `SUBSTRING(ags, 1, 5)` to rank the Top 5 fatal accident districts for 2024.
> 6. *(Switch to /docs tab)* → Finally, on `/docs`, examiners can test any endpoint interactively."

Slide 11: Key Technical Challenges & Decisions

`8:00 - 8:45`

Goal: Address engineering challenges and rubric compliance.

> "To highlight four key engineering decisions:
> 1. **8-Digit AGS Padding:** Standardized variable key lengths into 8-character strings via `.zfill()`.
> 2. **Foreign Key Guards:** Used `INSERT ... ON CONFLICT DO NOTHING` to prevent missing region keys from breaking ingestion.
> 3. **German Comma Sanitization:** Replaced German decimal commas with standard dots before parsing WGS84 floats.
> 4. **OPAL Compliance:** Used CDN styling and database SQL execution to keep the submission under 10 MiB without bundling heavy static JSON."

Slide 12: Summary & Future Enhancements

`8:45 - 9:30`

Goal: Summarize deliverables and propose future technical expansions.

> "In summary, the project successfully delivers:
> An automated open data ETL pipeline, a robust PostgreSQL relational database, a documented FastAPI REST backend, and a modern SPA UI meeting 100% of course requirements.
>
> In the future, this system can be expanded by adding interactive Leaflet.js GIS map heatmaps, multi-year temporal trend forecasting, and PostGIS spatial buffer queries to analyze accidents near schools."

Slide 13: Thank You!

`9:30 - 10:00`

Goal: Clean conclusion and invitation for examiner Q&A.

> "Thank you very much, Prof. Dr. Martin and Mr. Hahn, for your time and attention.
>
> I am now ready and welcome any questions regarding the ETL architecture, database schema, or REST API implementation."

💡 EXAMINER DEFENSE TIPS FOR SAMAY:

- **Speak clearly and naturally:** Maintain a calm, steady pace (around 130 words per minute).
- **Use your mouse cursor:** Point to key table rows on Slide 6 (Schema) and Slide 7 (API Table) as you mention them.
- **Live Demo Confidence:** Keep your browser tabs open on `http://127.0.0.1:8000/` and `http://127.0.0.1:8000/docs` before starting the presentation.